CLI-Based Network Traffic Monitor

A PROJECT REPORT

Submitted by

Shivam Yadav
Chandesh Patwari
Rupesh Singh
Vineet Shukla

in partial fulfillment for the award of the degree of

Bachelor of Technology

IN

Computer Science and Engineering

SET, MRIIRS

ABSTRACT

NetScope is a real-time network traffic analysis system developed using Python and low-level packet inspection techniques. The system captures live network packets, processes them to extract meaningful information, and displays analytics such as protocol distribution, traffic rate, active connections, and top communicating hosts.

The project integrates packet sniffing, protocol parsing, and statistical analysis into a lightweight command-line dashboard. It supports filtering based on protocol and port, detects application-layer services like HTTP, HTTPS, DNS, and QUIC, and provides insights into network behavior.

This project demonstrates practical applications of computer networking concepts such as packet structure, protocol analysis, and traffic monitoring using tools like Scapy.

1. INTRODUCTION

Modern networks generate massive volumes of data, making real-time traffic monitoring essential for performance analysis, debugging, and security.

NetScope is designed as a lightweight network packet analyzer that:

- Captures live traffic from a network interface
- Parses packets into structured data
- Performs real-time statistical analysis
- Displays results in a clean CLI dashboard

The system simulates the functionality of professional tools like Wireshark but in a simplified and programmable form.

2. OBJECTIVES

- To capture live network packets using low-level packet sniffing
- To parse packets and extract protocol-level information
- To classify traffic into services (HTTP, HTTPS, DNS, QUIC)
- To analyze traffic patterns such as:
    - Packet rate
    - Active connections
    - Top sources and destinations
- To provide filtering based on protocol and port
- To implement a real-time CLI-based monitoring dashboard

3. SYSTEM ANALYSIS

The system is designed as a **modular packet processing pipeline**.

Key Functionalities:

- Packet capturing using Scapy
- Packet parsing and protocol detection
- Traffic filtering (protocol/port)
- Real-time statistics computation
- Logging of observed traffic
- Payload inspection (hex dump)

Design Goals:

- Efficiency (minimal overhead)
- Real-time responsiveness
- Modularity (separate components)
- Scalability for future features

4. SYSTEM ARCHITECTURE

The project follows a **layered modular architecture**:

Modules:

1. **Sniffer** → Captures packets
2. **Parser** → Extracts structured data
3. **Analyzer** → Computes statistics
4. **Display (Main)** → CLI dashboard
5. **Logger** → Stores logs
6. **HostInfo** → Fetches system info
7. **Dump Tool** → Payload inspection

5. FILE STRUCTURE AND MODULE DESCRIPTION

1. main.py

- Entry point of the system
- Initializes analyzer and filters
- Runs:
    - Sniffer thread
    - Real-time dashboard

2. sniffer.py

- Uses Scapy's sniff() function
- Captures packets continuously
- Passes packets to parser

3. parser.py

- Extracts:
    - Source/Destination IP
    - Protocol (TCP/UDP/ICMP)
    - Ports
- Detects services:
    - HTTP (port 80)
    - HTTPS (port 443)
    - DNS (port 53)
    - QUIC (UDP 443)
- Extracts HTTP metadata (method, host)

4. analyzer.py

Core processing engine:

- Maintains:
    - Packet counters
    - Protocol statistics
    - Service statistics
    - Active connections

- Tracks:
    - Top sources/destinations (heap optimization)
    - Packet rate
- Stores recent packets (deque)

5. logger.py

- Logs unique packet summaries
- Writes to capture.log

6. hostinfo.py

- Retrieves:
    - Local IP address
    - MAC address

7. dump.py

- Specialized debugging tool
- Captures payload between two IPs
- Displays:
    - Hex dump
    - ASCII representation
- Detects direction (IN/OUT)

6. IMPLEMENTATION DETAILS

Packet Capture

- Implemented using **Scapy**
- Real-time sniffing without storage overhead

Packet Parsing

Each packet is converted into structured format:

```
{
 src, dst,
 protocol,
 sport, dport,
 service,
 method, host
}
```

Traffic Analysis

The analyzer computes:

- Protocol distribution (TCP, UDP, ICMP)
- Service-level traffic (HTTP, HTTPS, DNS, QUIC)
- Packet rate (packets/sec)
- Unique peers and connections
- Top communicating IPs

Optimization Techniques

- heapq.nlargest() for top-k queries
- Cached results to avoid repeated sorting
- deque for fixed-size recent packet storage

7. OUTPUT AND DASHBOARD

The CLI dashboard displays:

- Packet rate
- Protocol statistics
- Service statistics
- Host information (IP, MAC)
- Active connections and peers
- Top sources and destinations
- Recent packet activity

Example Output:

```
====== NetScope ======

Packets/sec: 120.5

TCP: 340   UDP: 210   ICMP: 5
HTTP: 50   HTTPS: 200   QUIC: 30   DNS: 80

====== HOST INFO ======

Local IP: 192.168.x.x
MAC: xx:xx:xx:xx:xx:xx

Top Destinations:
8.8.8.8 → 40 packets
```

```
====== NetScope ======

Packets/sec: 12.75

TCP: 36   UDP: 79   ICMP: 0
HTTP: 0   HTTPS: 35   QUIC: 79   DNS: 0

====== HOST INFO ======

Local IP: 10.250.37.2
MAC: 00:15:5d:b3:09:0d
Active Connections: 14
Unique Peers: 13

Top Destinations:
192.178.211.84 → 18 packets
172.64.41.3 → 10 packets
44.225.209.171 → 5 packets

Top Sources:
10.250.37.2 → 52 packets

192.178.211.84 → 22 packets

172.64.41.3 → 11 packets

Recent Packets:
[UDP  ] 10.250.37.2:60731 → 142.250.193.35:443
[TCP  ] 44.225.209.171:443 → 10.250.37.2:60910
[UDP  ] 142.250.193.35:443 → 10.250.37.2:60731
[TCP  ] 10.250.37.2:63083 → 216.239.38.223:443
[TCP  ] 216.239.38.223:443 → 10.250.37.2:63083
[TCP  ] 216.239.38.223:443 → 10.250.37.2:63083
[TCP  ] 216.239.38.223:443 → 10.250.37.2:63083
[TCP  ] 10.250.37.2:63083 → 216.239.38.223:443
[TCP  ] 216.239.38.223:443 → 10.250.37.2:63083
[TCP  ] 10.250.37.2:63083 → 216.239.38.223:443
```

8. RESULTS AND DISCUSSION

- HTTPS traffic dominates modern networks
- DNS traffic appears frequently due to domain resolution
- QUIC traffic is observed over UDP (modern web protocols)
- Some IPs act as hotspots (high communication frequency)
- Real-time monitoring helps identify unusual traffic
- Protocol distribution reflects application usage
- Connection tracking helps understand network behavior

9. CONCLUSION

NetScope successfully demonstrates a real-time network traffic analysis system using Python and packet-level programming.

It integrates:

- Packet sniffing
- Protocol parsing
- Statistical analysis
- CLI visualization

The project bridges theoretical networking concepts with practical implementation and provides a strong foundation for advanced network monitoring tools.

10. FUTURE ENHANCEMENTS

- GUI dashboard (using Tkinter)
- Packet filtering by IP ranges
- Intrusion detection system (IDS) integration
- Export reports (CSV/JSON)
- Deep packet inspection

11. REFERENCES

1. Scapy Documentation – https://scapy.net
2. Python Official Documentation – https://docs.python.org